Practice 10 — Docker in Production: Registry, Server, Release, Rollback

Objective: take last week's Flask app, harden the image, push it to a **registry**, run it on a **server** that only pulls (never builds) with Docker Compose — healthcheck, restart policy, log rotation, memory limit and a secret from a file — then release **v2**, **roll back** to v1, and put a **Trivy** security gate in front of it. These are the skills from today's lecture.

Timebox: ~55–60 min, in class. If you're stuck on one step for more than 10 minutes, ask instead of pushing through alone.

> **How you're checked:** this practice is done in class and **checked orally** at the end. There's no written report. Keep your terminal(s) open and be ready to **show** each item in the Definition of done and **explain** what each step does. The instructor will ask you a few of the questions under "Be ready to explain".

> **Two tracks, same skills. Track A — remote server:** push to **GitHub Container Registry (GHCR)** or Docker Hub and run the app on **any Linux VM you can SSH into** (a university VM, a free-tier VM from any cloud, or a local Multipass/VirtualBox VM). **Track B — laptop only:** a local registry on `localhost:5001` and a "server" folder on your laptop that only pulls. Both start with **Part 0**. Tell the instructor which track you did.

> **Before class (at home):** Docker working (`docker run hello-world`), and pull these once so class Wi-Fi isn't the bottleneck: `docker pull python:3.12-slim` , `docker pull registry:2.8.3` , `docker pull aquasec/trivy:0.69.3` .

Definition of done (things you can show)

□ Your **Dockerfile** with a non-root `USER` and a `HEALTHCHECK` , and `whoami` inside the running container printing `appuser`
□ The image **in the registry** with tags `v1` and `v2` (Track A: the package page on GHCR/Docker Hub; Track B: `/v2/lab10-app/tags/list`)
□ The app answering **from the server**, started by `docker compose` from an image it **pulled**, with status `(healthy)`
□ `docker inspect` proving the **restart policy**, **memory/CPU limit** and **log rotation** settings
□ `/secret-check` returning `"secret_loaded": true` — the secret came from a **file**, not from the image or `compose.yaml`
□ `/version` showing **v2** after the release and **v1** after the rollback; a note you saved **before** the release still there

□ Tag vs **digest** for v1 and v2, and which digest the server runs
□ The app **crashing** (`POST /crash`) and Docker **restarting** it
□ A **Trivy** scan: the gate failing, what you changed, and the gate passing (or the findings you accepted and why)
□ Cleanup done and shown

Part 0 — The app and a production-ready image (everyone, ~10 min)

Make a folder `lab10` with these files. It's last week's app plus `/healthz` (for the healthcheck), `/version` (from an env var), `/secret-check` (proves a secret file is mounted, never prints it) and `/crash` (to test restart policies).

`app.py` :

```
import os, socket
from flask import Flask

app = Flask(__name__)
DATA = os.environ.get("DATA_DIR", "/data")
VERSION = os.environ.get("APP_VERSION", "dev")
SECRET_FILE = os.environ.get("API_TOKEN_FILE", "/run/secrets/api_token")

@app.get("/")
def index():
    os.makedirs(DATA, exist_ok=True)
    return {"host": socket.gethostname(), "files": sorted(os.listdir(DATA))}

@app.get("/healthz")
def healthz():
    return {"status": "ok"}

@app.get("/version")
def version():
    return {"version": VERSION, "message": "Hello from lab10"}

@app.get("/secret-check")
def secret_check():
    # never return the secret itself, only whether it is there
    try:
        with open(SECRET_FILE) as f:
            return {"secret_loaded": True, "length": len(f.read().strip())}
    except FileNotFoundError:
        return {"secret_loaded": False}, 500

@app.post("/notes/<name>")
def add_note(name):
    os.makedirs(DATA, exist_ok=True)
    with open(os.path.join(DATA, f"{name}.txt"), "w") as f:
        f.write(f"note {name}\n")
    return {"saved": f"{name}.txt"}, 201

@app.post("/crash")
def crash():
    os._exit(1)          # simulate the app dying

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

`requirements.txt`:

```
flask==3.1.3
```

`.dockerignore`:

```
.venv/
__pycache__/
.env
secrets/
*.pem
```

`Dockerfile` — fill in the TODOs (all on the lecture slides):

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ARG APP_VERSION=dev
ENV APP_VERSION=$APP_VERSION DATA_DIR=/data
# TODO 1: create a user "appuser" with a home folder, create /data
#         and make appuser its owner (one RUN line, joined with &&)
# TODO 2: switch to that user
EXPOSE 8000
# TODO 3: a HEALTHCHECK every 10s (timeout 3s, 3 retries) that requests
#         http://localhost:8000/healthz. The image has no curl: use
#         python -c "import urllib.request; urllib.request.urlopen('...')"
CMD ["python", "app.py"]
```

Quick local test:

```
docker build --build-arg APP_VERSION=v1 -t lab10-app:v1 .
docker run --rm -d --name test -p 8001:8000 lab10-app:v1
curl -s localhost:8001/version     # {"message":"Hello from lab10","version":"v1"}
docker exec test whoami            # appuser
sleep 12; docker ps --filter name=test --format '{{.Status}}'   # Up … (healthy)
docker rm -f test
```

The "server" folder (both tracks)

The server never sees your source code. It only gets this folder:

```
server/
├── compose.yaml
├── .env                  # which image and tag to run — not secret
└── secrets/
    └── api_token.txt     # the secret — never in git, never in the image
```

`server/compose.yaml`:

```
services:
  app:
    image: ${IMAGE}:${TAG}
    restart: unless-stopped
    ports:
      - "${PORT:-8080}:8000"
    environment:
      API_TOKEN_FILE: /run/secrets/api_token
    secrets:
      - api_token
    volumes:
      - notes:/data
    mem_limit: 256m
    cpus: 0.5
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:8000/healthz')"]
      interval: 10s
      timeout: 3s
      retries: 3

secrets:
  api_token:
    file: ./secrets/api_token.txt

volumes:
  notes:
```

Create the secret (a random value — nobody needs to remember it):

```
mkdir -p server/secrets
openssl rand -hex 24 > server/secrets/api_token.txt
chmod 600 server/secrets/api_token.txt
```

> Compose (without Swarm) **bind-mounts** this file into the container. On a Linux server the container user `appuser` (uid **1000**) must be able to read it: if `/secret-check` says `secret_loaded: false`, run `sudo chown 1000:1000 secrets/api_token.txt` on the server.

Track A — remote server (GHCR or Docker Hub + any Linux VM)

A1. Log in to the registry and push a multi-arch v1

Create a GitHub **Personal access token (classic)** with `write:packages` (Settings → Developer settings). Never paste it into a file in the repo.

```
export GH_USER=<your-github-username>        # lower case
read -s CR_PAT                                # paste the token, Enter
echo "$CR_PAT" | docker login ghcr.io -u "$GH_USER" --password-stdin
# Login Succeeded

IMAGE=ghcr.io/$GH_USER/lab10-app
docker buildx create --use --name multi 2>/dev/null || docker buildx use multi
docker buildx build --platform linux/amd64,linux/arm64 \
  --build-arg APP_VERSION=v1 -t $IMAGE:v1 --push .
docker buildx imagetools inspect $IMAGE:v1   # one tag, two platforms
```

Why both platforms: your laptop may be `arm64` (Apple Silicon) while the server is `amd64` — or the other way round. One tag, the server pulls the right one.

On github.com → your profile → **Packages** → **lab10-app**: either make the package **public**, or keep it private and log in on the server in A3. (Docker Hub works the same: `docker login`, image `docker.io/<user>/lab10-app`.)

A2. Prepare the server

SSH into your VM (Ubuntu shown) and install Docker with the Compose plugin:

```
ssh <user>@<server-ip>
curl -fsSL https://get.docker.com | sudo sh    # official convenience script
sudo usermod -aG docker $USER && exit           # log out and back in
ssh <user>@<server-ip>
docker version && docker compose version
```

Allow only what you need through the VM's firewall / security rules: **SSH from your IP** and the **app port** (8080 here).

A3. Ship the server folder and start it

On your **laptop**:

```
printf 'IMAGE=%s\nTAG=v1\nPORT=8080\n' "$IMAGE" > server/.env
scp -r server <user>@<server-ip>:~/lab10
```

On the **server**:

```
cd ~/lab10
sudo chown 1000:1000 secrets/api_token.txt      # readable by appuser (uid 1000)
# private package only: a token with read:packages, not your write token
echo "<read-only token>" | docker login ghcr.io -u <github-username> --password-stdin
docker compose pull && docker compose up -d
sleep 15
docker compose ps                                # … Up … (healthy)
curl -s localhost:8080/version
curl -s localhost:8080/secret-check              # {"length":48,"secret_loaded":true}
curl -s -X POST localhost:8080/notes/first       # saved before the release
```

From your **laptop**: `curl -s http://<server-ip>:8080/version`.

Then do **Part C** on the server (use `sudo` only if your user isn't in the `docker` group).

A4. Release v2 and roll back

On your **laptop**, change the message in `app.py` (e.g. `"Hello from lab10, now with v2"`), then:

```
docker buildx build --platform linux/amd64,linux/arm64 \
  --build-arg APP_VERSION=v2 -t $IMAGE:v2 --push .
```

On the **server**:

```
sed -i 's/^TAG=.*/TAG=v2/' .env
docker compose pull && docker compose up -d     # recreates only the app
curl -s localhost:8080/version                  # v2, new message
curl -s localhost:8080/                         # "first.txt" is still there

sed -i 's/^TAG=.*/TAG=v1/' .env                 # roll back
docker compose up -d                            # v1 is cached: seconds
curl -s localhost:8080/version                  # v1 again
```

Track B — laptop only (local registry)

Track A	Track B
GHCR / Docker Hub	A local registry: `registry:2.8.3` on `localhost:5001`
`docker login ghcr.io`	Not needed (the local registry has no auth)
`ghcr.io/<user>/lab10-app:v1`	`localhost:5001/lab10-app:v1`
A Linux VM over SSH	The `server/` folder on your laptop, after deleting local images

Port **5001**, not 5000: on macOS, port 5000 belongs to AirPlay.

B1. Start the registry, push v1

```
docker run -d --name registry --restart unless-stopped -p 5001:5000 registry:2.8.3
curl -s localhost:5001/v2/_catalog                # {"repositories":[]}

R=localhost:5001/lab10-app
docker build --build-arg APP_VERSION=v1 -t $R:v1 .
docker push $R:v1                                 # v1: digest: sha256:… size: …
curl -s localhost:5001/v2/lab10-app/tags/list     # {"name":"lab10-app","tags":["v1"]}
```

B2. Be the server: pull, don't build

Delete the local copies, so Compose **must** pull from the registry:

```
docker rmi $R:v1 lab10-app:v1 2>/dev/null
cd server
printf 'IMAGE=localhost:5001/lab10-app\nTAG=v1\nPORT=8080\n' > .env
docker compose up -d                              # Image … Pulling … Pulled
sleep 15
docker compose ps                                 # … Up … (healthy)
curl -s localhost:8080/version                    # {"message":"Hello from lab10","version":"v1"}
curl -s localhost:8080/secret-check               # {"length":48,"secret_loaded":true}
curl -s -X POST localhost:8080/notes/first        # saved before the release
```

B3. Release v2, roll back

Change the message in `app.py` (e.g. `"Hello from lab10, now with v2"`), then from the `lab10` folder:

```
docker build --build-arg APP_VERSION=v2 -t $R:v2 .     # pip install: CACHED
docker push $R:v2
curl -s localhost:5001/v2/lab10-app/tags/list          # tags v1 and v2

cd server
sed -i.bak 's/^TAG=.*/TAG=v2/' .env                    # .bak keeps macOS sed happy
docker compose pull && docker compose up -d
curl -s localhost:8080/version                         # v2, new message
curl -s localhost:8080/                                # "first.txt" is still there

sed -i.bak 's/^TAG=.*/TAG=v1/' .env                    # roll back
docker compose up -d
curl -s localhost:8080/version                         # v1 again
```

Then do **Part C**.

Part C — Inspect, digests, crash, logs (both tracks, ~10 min)

From the `server` folder (Track A: on the server; set `R` to your image name, e.g. `R=ghcr.io/<user>/lab10-app`):

```
C=$(docker compose ps -q app)

# 1. the production settings really applied
docker inspect --format 'mem={{.HostConfig.Memory}} cpus={{.HostConfig.NanoCpus}} restart=
{{.HostConfig.RestartPolicy.Name}} log={{.HostConfig.LogConfig.Type}}
{{.HostConfig.LogConfig.Config}}' $C
#   mem=268435456 cpus=500000000 restart=unless-stopped log=json-file map[max-file:3 max-size:10m]
docker exec $C whoami                                   # appuser
docker exec $C ls -l /run/secrets/                      # api_token, mounted at runtime

# 2. tag vs digest
docker image inspect --format '{{index .RepoDigests 0}}' $R:v1 $R:v2
docker inspect --format '{{.Config.Image}} {{.Image}}' $C    # the image the server runs

# 3. crash test
curl -s -X POST localhost:8080/crash; echo " exit=$?"   # no reply, exit=52: the app died
sleep 4
docker inspect --format '{{.State.Status}} restarts={{.RestartCount}}' $C
#   running restarts=1

# 4. logs
docker compose logs --tail 5 app
```

The two digests are different; `v1` and `v2` never move. A tag like `latest` can be pushed again and point somewhere else — a **digest** can't.

Part D — Trivy security gate (both tracks, ~10 min)

Scan the image you are running. Trivy runs as a container and reads your local images through the Docker socket:

```
docker volume create trivy-cache
alias trivy='docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-
cache:/root/.cache aquasec/trivy:0.69.3'

trivy image --severity HIGH,CRITICAL --exit-code 1 $R:v1; echo "gate exit=$?"
```

Expect the gate to **fail** (exit 1): the Debian base image almost always has HIGH findings. Read the table: which **package**, is there a **Fixed Version**? Then:

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 $R:v1; echo "gate exit=$?"
```

`--ignore-unfixed` hides findings nobody can fix yet. If something fixable is left, fix it in the **Dockerfile** — e.g. add this line right after `FROM`, rebuild and scan again until the gate passes (exit 0):

```
RUN apt-get update && apt-get -y upgrade && rm -rf /var/lib/apt/lists/*
```

(When we checked on 2026-11-06: 48 HIGH in total, 2 fixable — both in `apt` — and the upgrade line made the gate pass. Your numbers will differ: new CVEs appear every day.)

Bonus: an SBOM — the list of everything inside the image:

```
trivy image --format cyclonedx $R:v1 > sbom.json
```

Stretch (optional) — the same app on Kubernetes

Needs a local cluster: the Kubernetes built into **Docker Desktop** or **OrbStack**, or **k3d** (k3s in Docker: `k3d cluster create lab10` then `k3d image import $R:v1 $R:v2 -c lab10`).

```
kubectl create namespace lab10
kubectl -n lab10 create deployment lab10 --image=$R:v1 --replicas=2
kubectl -n lab10 expose deployment lab10 --port=80 --target-port=8000
kubectl -n lab10 rollout status deployment/lab10
kubectl -n lab10 get pods                               # 2 × Running

kubectl -n lab10 scale deployment lab10 --replicas=3
kubectl -n lab10 set image deployment/lab10 lab10-app=$R:v2
kubectl -n lab10 rollout status deployment/lab10
kubectl -n lab10 run curl --image=curlimages/curl:8.11.1 --restart=Never --rm -i -- \
  curl -s http://lab10/version                          # v2

kubectl -n lab10 rollout undo deployment/lab10          # back to v1
kubectl -n lab10 rollout history deployment/lab10
kubectl delete namespace lab10                          # cleanup
```

Old pods keep answering for up to ~30 seconds while they terminate: if the first `curl` after `rollout undo` still says v2, wait a few seconds and try again. Compare: Compose replaced one container by hand; Kubernetes did a **rolling update** and keeps the history.

Cleanup

Track B (from the `server` folder, then `lab10`):

```
docker compose down -v                  # app + the notes volume
docker rm -f registry
docker rmi $R:v1 $R:v2 2>/dev/null
docker volume rm trivy-cache
```

Track A: on the server `docker compose down -v`; delete or stop the VM if it costs money; delete the `lab10-app` package on GHCR/Docker Hub if you don't need it; on your laptop `docker logout ghcr.io` and revoke the token you created today.

Be ready to explain

1. Registry, repository, tag, digest: what is each? Why is `v1` safer to deploy than `latest`?
2. Why does the server only **pull** and never build?
3. Why build `linux/amd64,linux/arm64`? What happens if a Mac-built `arm64`-only image is started on an `amd64` server?
4. What's the difference between the **healthcheck** and the **restart policy**? What did each do in your crash test?
5. Where does the API token live, and why not in the image, the `Dockerfile`, `compose.yaml` or `.env`?

6. What do `max-size` / `max-file` and `mem_limit` protect the server from?
7. Walk me through your release and rollback. Why was the rollback so fast, and why did the note survive both?
8. What did Trivy find, what did you fix, and what would you do with a finding that has no fix yet?

Common mistakes

- **`denied` / `unauthorized` on push.** Not logged in to that registry, the token lacks `write:packages`, or the image name isn't lower case (`ghcr.io/<user>/…` must be lower case).
- **`exec format error` on the server.** The image was built for the wrong CPU (arm64 on an amd64 server). Build multi-arch with `buildx --platform linux/amd64,linux/arm64`.
- **`multiple platforms feature is currently not supported for docker driver`.** Create a buildx builder first: `docker buildx create --use`.
- **The server "builds" instead of pulling.** Compose with an `image:` line and no `build:` always pulls; if you copied the source code to the server, you did it the old way.
- **Status stays `(health: starting)` or turns `(unhealthy)`.** The healthcheck URL or port is wrong, or the app isn't listening on `0.0.0.0`. Read `docker inspect --format '{{json .State.Health}}' <c>`.
- **`/secret-check` says `secret_loaded: false`.** The file isn't at `secrets/api_token.txt`, or on Linux `appuser` (uid 1000) can't read it: `sudo chown 1000:1000 secrets/api_token.txt`.
- **`sed -i` fails on macOS.** Use `sed -i.bak …` (Track B) or edit `.env` by hand.
- **The rollback "didn't work".** You changed `.env` but didn't run `docker compose up -d`, or you're curling a different port.
- **`docker compose down -v` deleted your notes.** `-v` removes named volumes. Plain `down` keeps them.
- **Token in shell history or in a file in git.** Use `read -s` and `--password-stdin`; revoke the token after class.